МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
КАФЕДРА ИИТ

Лабораторная работа №4

По дисциплине «Современный платформы программирования»

Выполнил:
Студент 3 курса
группы ПО-12:
Ефимович С.Л.
Проверил:
Кулик А.Д.

Брест 2026

Цель: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Ход работы

Вариант 5. Построение карты взаимодействий разработчика на GitHub

Условие:

Напишите Python-скрипт, который:

1. Запрашивает у пользователя имя пользователя GitHub.
2. Использует API GitHub для получения списка всех репозиториев, в которых этот пользователь:
   - Делал коммиты
   - Открывал pull requests
   - Создавал issues
   - Ставил звёзды
3. Для каждого репозитория находит других разработчиков, с которыми пользователь взаимодействовал (например, авторы коммитов, ревьюеры pull requests, авторы issues).
4. Создаёт граф связей между пользователем и другими разработчиками.
5. Визуализирует граф с помощью networkx и matplotlib:
   - Узлы (nodes) — пользователи
   - Рёбра (edges) — взаимодействия (коммиты, PR, issues, звёзды)
6. Сохраняет граф в формате PNG/JPEG.

Введите имя пользователя GitHub: octocat

Анализируем взаимодействия пользователя octocat...

Найдено 42 связанных разработчика.

Граф сохранён в github_network.json

Визуализация графа сохранена в github_network.png

Код программы:

```
import requests
import networkx as nx
import matplotlib.pyplot as plt
import json
import time

GITHUB_TOKEN = ""

headers = {}
```

```
if GITHUB_TOKEN:
   headers["Authorization"] = f"token {GITHUB_TOKEN}"


def get_repos(username):
   url = f"https://api.github.com/users/{username}/repos"
   repos = []
   page = 1

   while True:
       response = requests.get(
           url, headers=headers, params={"page": page, "per_page": 100}
       )
       data = response.json()

       if not data:
           break

       repos.extend(data)
       page += 1

   return repos


def get_contributors(repo_full_name):
   url = f"https://api.github.com/repos/{repo_full_name}/contributors"
   response = requests.get(url, headers=headers)
   return response.json()


def get_issues(repo_full_name):
   url = f"https://api.github.com/repos/{repo_full_name}/issues"
   response = requests.get(url, headers=headers)
   return response.json()


def get_pulls(repo_full_name):
   url = f"https://api.github.com/repos/{repo_full_name}/pulls"
   response = requests.get(url, headers=headers)
   return response.json()


def analyze_user(username):
   print(f"Анализируем взаимодействия пользователя {username}...")

   repos = get_repos(username)

   G = nx.Graph()
   G.add_node(username)

   interactions = set()

   for repo in repos:
       repo_name = repo["full_name"]

       print(f"Обрабатываем репозиторий: {repo_name}")

       contributors = get_contributors(repo_name)
       for user in contributors:
```

```
            if user["login"] != username:
                interactions.add(user["login"])
                G.add_edge(username, user["login"], type="commit")

        issues = get_issues(repo_name)
        for issue in issues:
            if "user" in issue and issue["user"]:
                login = issue["user"]["login"]
                if login != username:
                    interactions.add(login)
                    G.add_edge(username, login, type="issue")

        pulls = get_pulls(repo_name)
        for pr in pulls:
            if "user" in pr and pr["user"]:
                login = pr["user"]["login"]
                if login != username:
                    interactions.add(login)
                    G.add_edge(username, login, type="pr")

        time.sleep(0.5)

    print(f"Найдено {len(interactions)} связанных разработчиков.")

    return G


def save_graph(G):
    data = nx.readwrite.json_graph.node_link_data(G)
    with open("github_network.json", "w") as f:
        json.dump(data, f, indent=4)

    print("Граф сохранён в github_network.json")

    plt.figure(figsize=(12, 12))
    pos = nx.spring_layout(G)

    nx.draw(G, pos, with_labels=True, node_size=800, font_size=14)

    plt.title("GitHub Interaction Network")
    plt.savefig("github_network.png")
    plt.close()

    print("Визуализация сохранена в github_network.png")


if __name__ == "__main__":
    username = input("Введите имя пользователя GitHub: ")
    graph = analyze_user(username)
    save_graph(graph)
```

Результат работы программы:

```
Введите имя пользователя GitHub: SvyatProgrammer
Анализируем взаимодействия пользователя SvyatProgrammer...
Обрабатываем репозиторий: SvyatProgrammer/CookieClicker
Обрабатываем репозиторий: SvyatProgrammer/spp_po12
Обрабатываем репозиторий: SvyatProgrammer/StudentAttend
Обрабатываем репозиторий: SvyatProgrammer/SvyatProgrammer
Обрабатываем репозиторий: SvyatProgrammer/University-project
Обрабатываем репозиторий: SvyatProgrammer/VRHumanoidControl
Найдено 10 связанных разработчиков.
Граф сохранён в github_network.json
Визуализация сохранена в github_network.png

Process finished with exit code 0
```

Рисунок 1. Результат выполнения программы

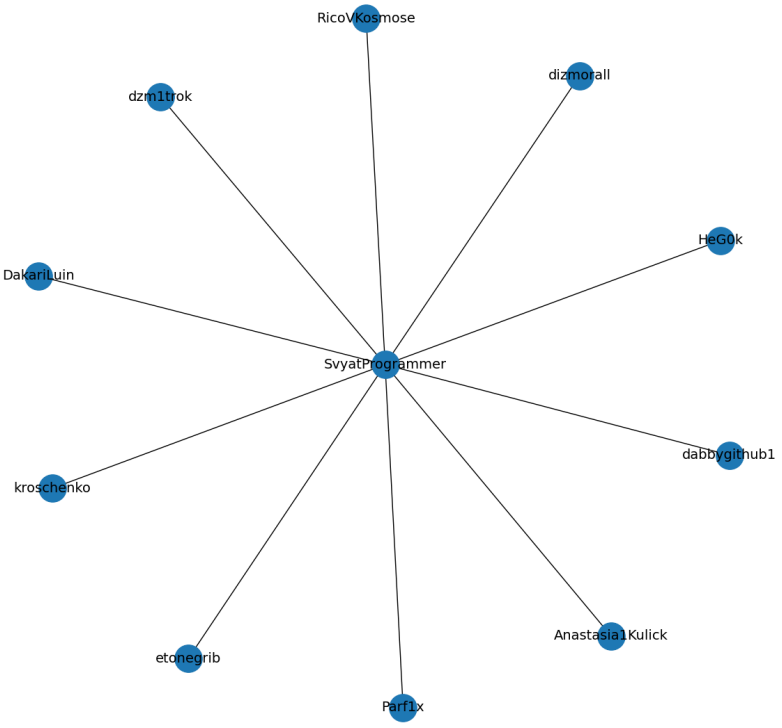

Рисунок 2. граф связей между пользователем и другими разработчиками.

Вывод: научился работать с Github API, приобрел практические навыки написания программ для работы с REST API или GraphQL API